

Provider Identity Without Provider Keys

The key that answers every frame on port 3333 should not be the key that can unstake you.

A Morpheus provider runs one secp256k1 key. `proxy-router` loads it from `WALLET_PRIVATE_KEY` or the OS keychain (`config.go:48`) and uses it for three unrelated jobs: holding stake, pricing bids, and signing every morrpc frame and receipt on a TCP port that must be reachable from the open internet. Those three jobs have different lifetimes and different blast radii. A traffic key should be minutes old and disposable; a stake key should never sit on a host that accepts inbound connections.

You already built the indirection

`SessionRouter._isValidProviderReceipt` accepts a receipt signed by the provider address, by a delegate holding `DELEGATION_RULES_PROVIDER`, or, when the provider is a contract, by its `owner()` (`SessionRouter.sol:537-559`). `proxy-router` mirrors it: `OnChainProviderAuthResolver` calls `owner()` on the consumer's own RPC, keyed by the address the consumer chose, so it cannot be forged in transit. A keyless provider is already a supported shape; this proposal finishes it.

Mechanism

The provider address becomes a custody contract whose `owner()` is a threshold key in a KMS, offline from every node. Stake authority lives there and nowhere else. Bid authority delegates separately: `DELEGATION_RULES_MARKETPLACE` already gates `postModelBid` and `deleteModelBid` (`Marketplace.sol:68,119`), so a warm key that reprices but cannot unstake is one delegation away.

Traffic authority is the right that does not exist yet. `_isValidProviderReceipt` accepts `DELEGATION_RULES_PROVIDER`, the same right that gates `providerRegister` and `providerDeregister` (`ProviderRegistry.sol:30,62`), so letting a datacenter sign receipts also lets it deregister you: forced delisting, not theft, since withdrawal returns to the provider address, but still the wrong radius for a key on a box with `:3333` open. Split it and rotation stops meaning re-registration: rotating or revoking a traffic key becomes a delegation write, leaving address, endpoint, bids and stake untouched, and consumers pick up the new signer on their next resolve.

Custody keys should be t -of- n , not a file. Lux's LSS rotates the holders without changing the public key, over two kernels: Lens on secp256k1, which is what Base requires since `ECDSA.recover` is the diamond's only verifier, and Pulsar, a lattice kernel whose combined output is byte-equal to a single-party FIPS 204 ML-DSA-65/87 signature. Pulsar is review-ready, not production-certified, and does not verify on Base; it is the path for custody that must outlive secp256k1. Lens buys the property today: no machine holds a whole key.

What a provider gets

- A staking key that never sits on a host with an open port.
- Rotation and revocation without re-registering: no endpoint churn, no rebuilt bids.
- A node operator, or a datacenter, that serves traffic but cannot unstake you.

What MOR holders get

- Provider compromise stops being a stake event. A stolen traffic key is good only inside the ten-minute `SIGNATURE_TTL` (`SessionStorage.sol:35`), each approval single-use.
- An on-chain record of which key held which right.

First step, and who takes it

One constant and one line, in facets Morpheus already owns: add `DELEGATION_RULES_RECEIPT` to `DelegationStorage` and have `_isValidProviderReceipt` check it instead of `DELEGATION_RULES_PROVIDER`. Existing EOA providers, the `owner()` branch and `provider_auth.go` are unaffected. A core contributor lands it under the existing 5-of-9; we write the custody contract, KMS adapter and tests.

While that is in flight, fix the port checker. `MyProvider` is the entire supply-side onboarding path, and its check reads a `TypeError` from `fetch` as proof the port is open (`apiService.ts:470-477`). A fetch to a closed port fails exactly that way, and `:3333` speaks morrpc, not HTTP, so no browser can check it. Operators stake MOR to advertise endpoints nothing can reach. The fix is a server-side probe completing one morrpc handshake.

What this does not solve. Nothing here proves inference happened, or that the advertised model served it. Custody bounds who may act as a provider, not what it returns. That is LP-313. LP-318, full paper available.